

CYBERSECURITY INTERNSHIP

Organization name: Elevate labs

TASK - 4: Password Security & Authentication Analysis

Password Storage Techniques

1. Plain Text Storage

Storing passwords in plain text is extremely dangerous. If a database is compromised, attackers can immediately view all user passwords.

2. Encrypted Password Storage

Encrypted passwords can be decrypted using a secret key. If the encryption key is stolen, all passwords become readable.

3. Hashed Password Storage

Hashed passwords:

- Cannot be reversed
- Protect original passwords even if the database is leaked
- Are the industry standard for password storage

Hashing Algorithms types:

MD5

- Produces a 128-bit hash
- Very fast, making it vulnerable to brute force attacks
- Considered completely insecure today

SHA-1

- Produces a 160-bit hash
- Vulnerable to collision attacks
- No longer recommended for password storage

bcrypt

- Adaptive hashing algorithm
 - Includes automatic salting
 - Adjustable cost factor increases resistance to attacks
 - Widely used in modern applications
-

Hash Identification Process

Hash identification was performed using:

- Hash length
- Character pattern
- Online hash identification tools

This step is crucial because the correct hash type must be known before attempting any cracking method.

Password Hash Generation

Sample passwords were hashed using different algorithms.

Observations:

- Same passwords produce different hashes across algorithms
 - Minor changes in passwords cause major hash differences
 - Strong algorithms significantly increase attack difficulty
-

Password Attack Techniques:

Dictionary Attacks:

Dictionary attacks rely on:

- Common passwords
- Leaked password databases
- Human behavioral patterns

These attacks are fast and effective against weak passwords.

Brute Force Attacks:

Brute force attacks:

- Try every possible character combination
- Are computationally expensive
- Become impractical for long passwords

Longer passwords exponentially increase cracking time.

Hybrid Attacks

Hybrid attacks combine:

- Dictionary words
- Character substitutions (e.g., P@ssw0rd)

These attacks are very effective against “slightly modified” passwords.

Practical Tool Analysis:

1. Hashcat

- Supports CPU and GPU acceleration
- Extremely fast compared to other tools
- Suitable for large-scale password auditing

2. John the Ripper

- Simple to use
- Good for learning and testing
- Supports multiple attack modes

Both tools clearly demonstrate the importance of strong passwords.

Why Weak Passwords Are Easily Cracked

Weak passwords fail due to:

- Short length
- Use of predictable words
- Keyboard patterns
- Reuse across multiple accounts

Attackers focus more on psychology than technology.

Role of Salting

Salting:

- Adds random data to passwords before hashing
- Prevents rainbow table attacks
- Makes identical passwords unique in hash form

Salting significantly improves password security.

Authentication Beyond Passwords

Multi-Factor Authentication (MFA)

MFA requires multiple authentication factors:

1. Knowledge factor (password)
2. Possession factor (OTP, security key)
3. Inherence factor (biometrics)

MFA greatly reduces the impact of password compromise.

Real-World Cyber Attack Scenarios

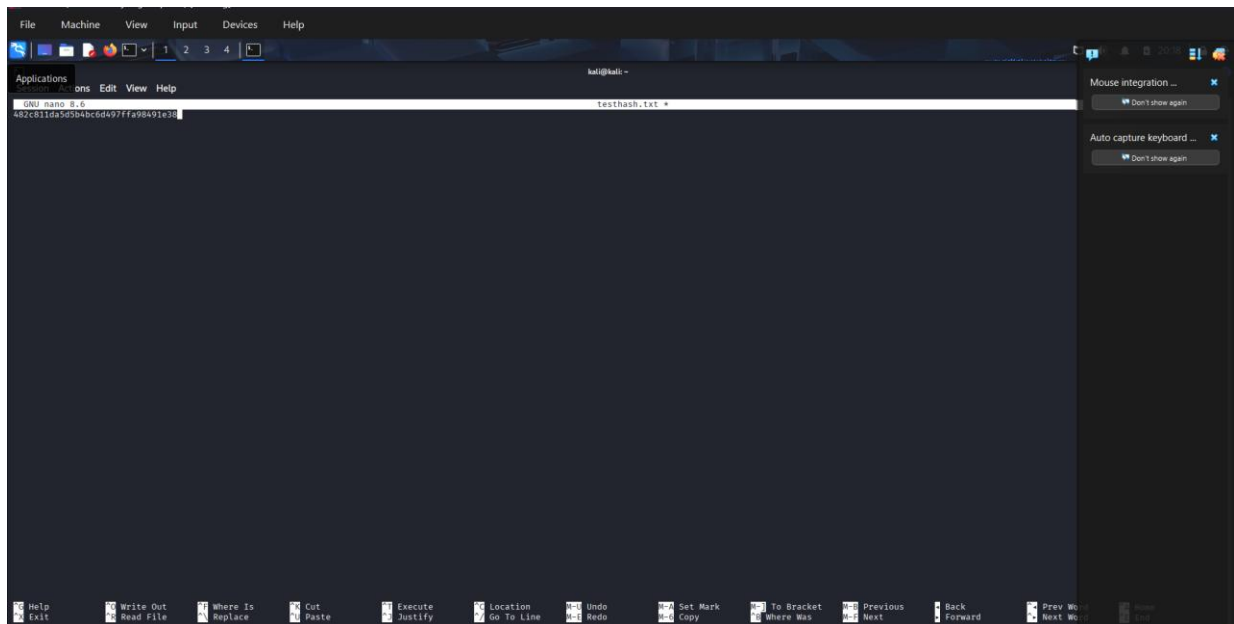
Many major cyber incidents occur due to:

- Weak password policies
- No MFA implementation
- Outdated hashing algorithms

Organizations that enforce strong password policies experience fewer security breaches.

Recommendations for Strong Authentication:

Strong authentication plays a key role in protecting user accounts. Using long and difficult passwords makes it much harder for attackers to break in. It is important not to reuse the same password across different platforms, because if one account is compromised, others can be affected too. Storing passwords with secure hashing methods such as bcrypt helps keep them safe. Enabling multi-factor authentication for important accounts adds an extra level of security. Password managers make it easier to create and manage strong passwords, and educating users about safe password habits is essential for better online security.



```
File Machine View Input Devices Help

Session Actions Edit View Help

(kali@kali)-[~]
$ echo -n password123 | md5sum
482c811da5d5b4bc6d497ffa98491e38 -

(kali@kali)-[~]
$ echo -n password132 | md5sum
fafda651eae8e1453edd6ac90b3adec -

(kali@kali)-[~]
$ echo -n password123 | sha1sum
cbfdac6008f9cab4083784cbd1874f76618d2a97 -

(kali@kali)-[~]
$ echo -n password132 | sha1sum
a8d607478b9ee391c39097348ce50cc878946d01 -

(kali@kali)-[~]
$ echo -n password123 | sha256sum
ef92b778baf6771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f -

(kali@kali)-[~]
$ echo -n password132 | sha256sum
376cd0595988f48b61e91a47e1c7e78cb2e42f73aa984c49851baad93548789e -

(kali@kali)-[~]
$
```

```
Session Actions Edit View Help

(kali@kali)-[~]
$ ls
Desktop Documents Downloads Music nipe packets_wireshark.pcapng Pictures Public Templates testhash.txt Videos

(kali@kali)-[~]
$ mv testhash.txt.save testhash.txt
mv: cannot stat 'testhash.txt.save': No such file or directory

(kali@kali)-[~]
$ mv testhash.txt.save testhash.txt
mv: cannot stat 'testhash.txt.save': No such file or directory

(kali@kali)-[~]
$ sudo gzip -d /usr/share/wordlists/rockyou.txt.gz
[sudo] password for kali:

(kali@kali)-[~]
$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt testhash.txt
john --show testhash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
password123 (?)
1g 0:00:00:00 DONE (2026-01-20 20:30) 16.66g/s 25600p/s 25600c/s 25600c/s 753951..mexico1
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
0 password hashes cracked, 2 left

(kali@kali)-[~]
$
```